CS1200: Introduction to Algorithms and their Limitations

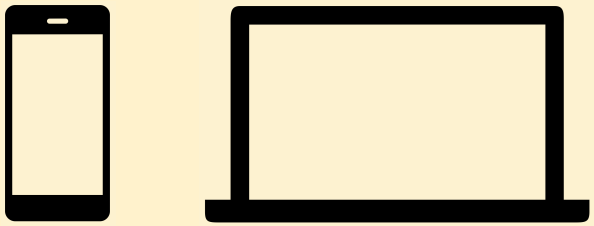
Lecture 1

September 2, 2026

Getting Around CS1200

- Course Site(s):
  - https://harvard-cs-1200.github.io/cs1200/
  - https://canvas.harvard.edu/courses/169766
- Syllabus
- Course calendar
- Getting help: TFs, Ed Discussion, Office Hours, FAQ, email
- Attendance and Participation

Please put away devices



AI and CS1200 (and you)

- Our shared goal is your learning. We want you to be able to use AI in ways that support your learning and be able to resist using AI when it would be an obstacle to learning.
- We have tried to structure the class so that unproductive uses of AI will also harm your success on course assignments. We will have in-class assessments including midterms, SREs, problem set follow-ups, and a final exam.
- We ask that you:
  - do not use AI or outside assistance on any in-class assessment.
  - do not use AI to solve homework problems or write-up any part of your homework.
  - consider writing your problem sets by hand
  - attend office hours without your devices (out) so that you can collaborate with peers and TFs
  - ask us about any uses of AI that you want to make and are unsure about
- We will be incorporating AI-generated feedback into our Gradescope auto graders this semester. This is for the purpose of giving you more extensive feedback and more consistent grading. You will always be able to see what feedback is AI-generated. Each problem on which there is AI-generated feedback will be manually reviewed and confirmed/corrected by a human grader.

What is an algorithm?

CS1200 At a Glance

- Unit 1: Storing and Search
  - Abstract and precisely mathematically specify a computational problem and algorithm
  - Prove that an algorithm solves a problem
  - Analyze and compare efficiency of algorithms
- Unit 2: Computational Models
  - Formalize what an algorithm is
- Unit 3: Graph and Logic Algorithms
- Unit 4 & 5: Computability and Complexity

Google Search

- Input: a keyword. Output: a list of URLs.
- Steps:
  1. **PageRank.** For every url on the web, compute $PR(url) \in [0,1]$.
  2. **Keyword search.** Given keyword w, compute $S_w = \{url : w \text{ appears on the page at } url\}$.
  3. **Sorting.** Return the urls in S_w in decreasing order of PageRank.

The Sorting Problem

- Input: $A = ((K_0, V_0), \ \ldots, \ (K_{n-1}, V_{n-1})), \qquad K_i \in \mathbb{R}$
- Output: $A' = ((K'_0, V'_0), \ \ldots, \ (K'_{n-1}, V'_{n-1}))$ such that A' is a valid sort of A
- What does it mean to be a valid sort?
  - $K'_0 \leq K'_1 \leq \cdots \leq K'_{n-1}$
  - There exists a permutation $\pi : [n] \to [n]$ such that $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$

Google Search #3 as Sorting Problem

- State step 3 of the Google Search algorithm as an instance of Sorting Problem
  - What are the keys? $K_i = ?$
  - What are the values? $V_i = ?$

Google Search #3 as Sorting Problem

- State step 3 of the Google Search algorithm as an instance of Sorting Problem
  - What are the keys?
    - $K_i = 1 - PR(url_i)$
  - What are the values?
    - $V_i = url_i$

Exhaustive-Search Sort

Input: A, Output: A' which is a valid sort of A (if it exists)

1. $ESS(A)$:
2. for each permutation $\pi : [n] \to [n]$:
3. if $K_{\pi(0)} \leq K_{\pi(1)} \leq \cdots \leq K_{\pi(n-1)}$:
4. return $A' = ((K_{\pi(0)}, V_{\pi(0)}), \ \ldots, \ (K_{\pi(n-1)}, V_{\pi(n-1)}))$
5. return $\perp$

Exhaustive-Search Sort Example

$A = ((6,a),(1,b),(6,c),(9,d))$

π(0)	π(1)	π(2)	π(3)	keys	sorted?
0	1	2	3	6,1,6,9	✗
0	1	3	2	6,1,9,6	✗
0	2	1	3	6,6,1,9	✗
0	2	3	1	6,6,9,1	✗
0	3	1	2	6,9,1,6	✗
0	3	2	1	6,9,6,1	✗
1	0	2	3	1,6,6,9	✓

$A' = ((1,b),(6,a),(6,c),(9,d))$

ESS Proof of Correctness

- Theorem. For every input array A, $ESS(A)$ returns a valid sorting of A if one exists.
- Proof. Let $A = ((K_0, V_0), \ \ldots, \ (K_{n-1}, V_{n-1})), \qquad K_i \in \mathbb{R}$.
- Suppose A has a valid sorting. Then some permutation π^*satisfies $K_{\pi^*(0)} \leq K_{\pi^*(1)} \leq \cdots \leq K_{\pi^*(n-1)}$
- The loop enumerates every permutation of $[n]$, so it reaches π^* unless it has already returned another satisfying permutation.
- When it returns A', line 3 checked that the keys ascend. Also A' was built from a permutation π. So A' is a valid sorting. ∎

Insertion Sort

Input: A, Output: A' which is a valid sort of A (if it exists)

1. $IS(A)$:
2. for $i = 0,\ldots,n-1$:
3. find the first index $j \leq i$ such that $A[i][0] \leq A[j][0]$
4. insert $A[i]$ at position j, shifting $A[j \ldots i-1]$ to positions $j+1,\ldots,i$
5. return A

Insertion Sort

Input: A, Output: A' which is a valid sort of A (if it exists)

1. $IS(A)$:
2. for $i = 0,\ldots,n-1$:
3. find the first index $j \leq i$ such that $A[i][0] \leq A[j][0]$
4. insert $A[i]$ at position j, shifting $A[j \ldots i-1]$ to positions $j+1,\ldots,i$
5. return A

Our notation for referring to key-value pairs changed. What's happening here?

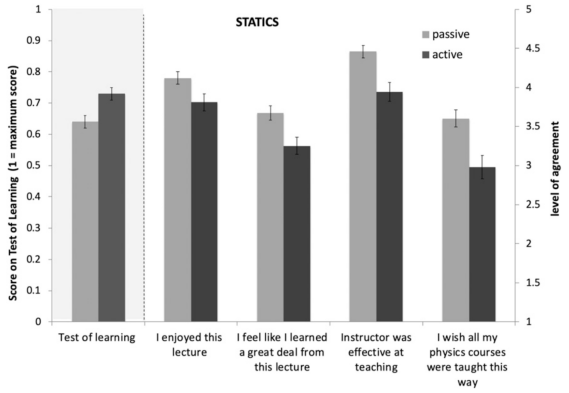
What would happen if we specified $A[i][0]$ and $A[j \ldots i-1][0]$ instead?

What would happen if we changed these to $<$? Would this be a good idea?

PNAS

Measuring actual learning versus feeling of learning in response to being actively engaged in the classroom

Louis Deslauriers[a,1], Logan S. McCarty[a,b], Kelly Miller[c], Kristina Callaghan[a], and Greg Kestin[a]



Insertion Sort

Input: A, Output: A' which is a valid sort of A (if it exists)

1. $IS(A)$:
2. for $i = 0,\ldots,n-1$:
3. find the first index $j \leq i$ such that $A[i][0] \leq A[j][0]$
4. insert $A[i]$ at position j, shifting $A[j\ldots i-1]$ to positions $j+1,\ldots,i$
5. return A

Our notation for referring to key-value pairs changed. What's happening here?

What would happen if we specified $A[i][0]$ and $A[j\ldots i-1][0]$ instead?

What would happen if we changed these to $<$? Would this be a good idea?

Insertion Sort Example

$A = ((6,a),(1,b),(6,c),(9,d))$

after i	array
0	(6,a), (1,b), (6,c), (9,d)
1	(1,b), (6,a), (6,c), (9,d)
2	(1,b), (6,c), (6,a), (9,d)
3	(1,b), (6,c), (6,a), (9,d)

$A' = ((1,b),(6,a),(6,c),(9,d))$

IS Proof of Correctness

- Theorem. For every input A, $IS(A)$ returns a valid sorting of A.
- Notation. Let $A^{(i)}$ be the array state just before iteration i. Note that $A^{(0)} = A$.
- We say $A^{(i)}$ satisfies loop invariant $P(i)$ if:
  - $A^{(i)}[0\ldots i-1]$ is sorted by key, and
  - $A^{(i)}[i\ldots n-1] = A[i\ldots n-1]$ (the suffix is untouched)
- We prove insertion sort preserves by $P(i)$ induction on i, for $i = 0,\ldots,n$. Note that $P(n)$ proves that $A^{(n)}$ is a valid sort of A.
- Base case $P(0)$. The prefix $A^{(0)}[0\ldots -1]$ is empty, hence vacuously sorted and $A^{(0)} = A$, so the suffix is unchanged. ✓

IS Proof of Correctness (cont.)

- Inductive hypothesis: Assume $P(i)$ for some $i < n$.
- Inductive Step: We show $P(i+1)$. Iteration i finds the first index j with $A[i][0] \leq A[j][0]$, inserts $A[i]$ at position j, and shifts $A[j\ldots i-1]$ up one.
  - We know such a j exists: $j = i$ always qualifies, since $K_i \leq K_i$.
  - Because j is the FIRST such index, every $j' < j$ has $K_{j'} < K_i$.
  - By $P(i)$ the prefix is sorted, so positions $j\ldots i-1$ all have keys greater than K_i.
- So after the insertion the prefix is $A^{(i)}[0\ldots j-1] + A[i] + A^{(i)}[j\ldots i-1]$ which is sorted. Positions $i+1\ldots n-1$ were not touched. Hence $P(i+1)$. ✓
- $P(n)$ holds. Every operation was a shift or a reinsertion, so A^(n) is a reordering of A. The output is a valid sort. ∎

Conclusion

- If we didn't get to it, continue on your own with MergeSort.
- Access the textbook. Read sections 1.1-1.4 on Perusall. Try to take each step in the merge sort example before you read how to do it.
- PS0 due September 8th.
- Look to the calendar and on Ed Discussion for announcements about office hours and sections, etc.

Merge Sort

Input: A, Output: A' which is a valid sort of A (if it exists)

1. $MS(A)$:
2. if $n \leq 1$: return A
3. else:
4. $i = \lceil n/2 \rceil$
5. $B = MS(A[0 \ldots i-1])$
6. $B' = MS(A[i \ldots n-1])$
7. return $Merge(B, B')$

Merge

Input: Arrays B (length n), B' (length n') already sorted. Output: a valid sorting of $B + B'$.

1. $Merge(B, B')$:
2. $i = i' = 0$ (indices for B, B'); $j = 0$ (index into output)
3. allocate C of length $n + n'$
4. while $j < n + n'$:
5. if $i' = n'$ or $B[i][0] \leq B'[i'][0]$: // B' is exhausted, or B's next key is smaller
6. $C[j] = B[i]; \quad i = i + 1; \quad j = j + 1$
7. else: $C[j] = B'[i']; \quad i' = i' + 1; \quad j = j + 1$
8. return C

Mergesort Example

$A = ((6,a), (1,b), (6,c), (9,d))$

step	state
input	(6,a), (1,b), (6,c), (9,d)
split	(6,a), (1,b) | (6,c), (9,d)
split	(6,a) | (1,b) | (6,c) | (9,d)
merge	(1,b), (6,a) | (6,c), (9,d)
merge	(1,b), (6,a), (6,c), (9,d)

$A' = ((1,b), (6,a), (6,c), (9,d))$

MS Proof of Correctness

Theorem. For every A, $MS(A)$ returns a valid sorting of A.

1. Let $n = |A|$. Proof by strong induction on n. Let $P(n)$ be the proposition: "MS correctly sorts arrays of size n."
2. Base cases $P(0), P(1)$. For $n \leq 1$, $MS(A) = A$; Every array of length 0 or 1 is sorted. ✓
3. Inductive step. Assume $P(k)$ for all $k < n$; take $n \geq 2$.
4. Let $i = \lceil n/2 \rceil$ and split A into $C = A[0 \ldots i-1]$ and $C' = A[i \ldots n-1]$. $0 < \lceil n/2 \rceil < n$, so $|C| < n$ and $|C'| < n$
5. By the inductive hypothesis, $B = MS(C)$ and $B' = MS(C')$ are valid sortings of C and C'.
6. By the Lemma (left as exercise), $Merge(B, B')$ is a valid sorting of $B + B'$; and $B + B'$ is a reordering of $C + C' = A$. So $Merge(B, B')$ is a valid sorting of A. ∎